

Aula 1 — O que é um Compilador? O Modelo Análise-Síntese

Compiladores I

FACOM - Faculdade de Computação

Bloco 1: Introdução à Compilação

Duração estimada: 70–80 minutos

Objetivos de aprendizagem

Ao final desta aula, você será capaz de:

- ▶ Definir o que é um compilador e diferenciá-lo de um interpretador;
- ▶ Explicar o modelo análise-síntese de compilação;
- ▶ Justificar a relevância do estudo de compiladores na formação em Computação.

Sumário da aula

Motivação

O modelo análise-síntese

Desafio e síntese

Compiladores estão em todo lugar

- ▶ Todo programa que você já executou passou por um compilador ou interpretador.
- ▶ Compiladores modernos (GCC, LLVM/Clang) têm milhões de linhas de código e décadas de otimizações — verdadeiras **caixas-pretas**.
- ▶ Neste curso usaremos o **Micro C**: um compilador didático, com menos de 4 mil linhas, que gera Assembly x86-64 a partir de um subconjunto da linguagem C.

Pergunta rápida

Você já usou um compilador hoje? Cite um exemplo de linguagem compilada e uma interpretada que você conhece.

Compilador x Interpretador

Compilador

- ▶ Traduz *todo* o código-fonte antes da execução;
- ▶ Gera um executável independente (ex: a.out);
- ▶ Execução final é rápida, mas depende da arquitetura/SO alvo;
- ▶ Exemplos: C, C++, Micro C.

Interpretador

- ▶ Traduz e executa instrução por instrução, em tempo real;
- ▶ Não gera executável prévio;
- ▶ Mais portátil, porém mais lento;
- ▶ Exemplos: Python, Ruby.

Pergunta rápida

Por que um programa compilado geralmente executa mais rápido do que um interpretado?

Sumário da aula

Motivação

O modelo análise-síntese

Desafio e síntese

O modelo análise-síntese

Duas grandes etapas de um compilador

- ▶ **Análise (front-end):** entende o código-fonte e verifica se ele é válido (léxica, sintática, semântica);
- ▶ **Síntese (back-end):** constrói o programa equivalente na linguagem-alvo (geração de código intermediário e final).

A análise cria uma representação intermediária do programa; a síntese constrói o programa-alvo a partir dessa representação.

O Micro C como linha de montagem

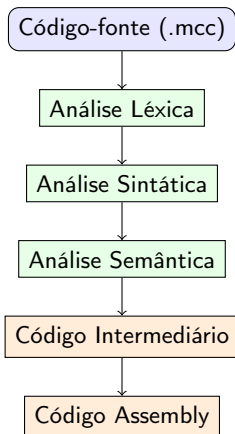
O compilador do Micro C divide o processo em 5 etapas visíveis e inspecionáveis:

1. Análise Léxica → tokens
2. Análise Sintática → Árvore Sintática Abstrata (ASA)
3. Análise Semântica → tabela de símbolos + verificação de tipos
4. Geração de Código Intermediário → Código de Três Endereços (CTE)
5. Geração de Código Final → Assembly x86-64

Exemplo — Por que isso importa

Diferente do GCC, o Micro C permite gerar e inspecionar o resultado de *cada* etapa separadamente (lista de tokens, ASA, tabela de símbolos, CTE, Assembly) — ideal para aprender.

Visualizando o fluxo de compilação



Sumário da aula

Motivação

O modelo análise-síntese

Desafio e síntese

Desafio em sala

Desafio em sala

Em grupos de 2–3 pessoas: escolham uma linha de código simples (ex: `int x = 2 + 3;`) e discutam quais informações cada etapa (léxica, sintática, semântica, código intermediário, Assembly) precisaria extrair ou produzir a partir dela. Anotem em uma frase por etapa.

Síntese da aula

- ▶ Compiladores traduzem antes de executar; interpretadores traduzem durante a execução;
- ▶ O modelo análise-síntese organiza o compilador em front-end (análise) e back-end (síntese);
- ▶ O Micro C divide esse processo em 5 fases inspecionáveis, que estudaremos ao longo do curso.

Próxima aula: fases do compilador em detalhe e especificação de linguagens de programação.

Teste rápido — Aula 1

Questões teóricas

1. Explique, com suas palavras, a diferença entre compilador e interpretador.
2. O que caracteriza a etapa de *análise* e a etapa de *síntese* em um compilador?

Questão prática

3. Para o trecho `float media = (a + b) / 2;`, liste, em ordem, as 5 etapas do Micro C que esse código atravessaria até virar Assembly.